ther, older Fieldnotes which are relevant to your current investigation, so try searching through your older notes and labels for ideas and code

Tips and tricks

- We have a Zendesk trigger which periodically reminds Support Engineers to add a ticket summary, and a macro for outlining the summary as a Public Comment. The default Fieldnote issue description template is based on this, with the intention that the issue description can be simply copied to the ticket whenever such summaries make sense

- Keep the Description up-to-date

- We have more than one description template for Fieldnotes: chose one to suit your need, and add more of your own

- Fieldnotes created with the Zendesk Fieldnote app will use the fieldnote.md template, but you can change templates if another would better suite the ticket

- Use browser bookmarks with keywords. Here are some suggestions to make searching for Fieldnote and GitLab issues easier:

keyword	URL
fn	https://gitlab.com/gitlab-com/support/fieldnotes/
fni	https://gitlab.com/gitlab-com/support/fieldnotes/-/issues/%s
fnm	https://gitlab.com/gitlab-com/support/fieldnotes/-/issues/?sort=createddate&state=opened&assigneeusername[]=auser
gl	https://gitlab.com/gitlab-org/gitlab
gli	https://gitlab.com/gitlab-org/gitlab/-/issues/%s

- Remember GitLab keyboard shortcuts:

Key sequence	What it does
/	searches within GitLab for a string
?	show keyboard shortcuts
i	creates a new Issue
e	Edit issue Description
r	Reply (add a comment)
r · (up arrow)	Reply to thread
l	change Labels
g i	Goes to the Issues
g f	Goes to the Files
g s	Goes to the Snippets

- Use the Fieldnotes issue board. In it you can see fieldnotes grouped by stage or interesting collaboration labels.

- Remember that Tickets have one Assignee, who is the DRI, but Issues can have multiple Assignees who collaborate

Handy browser plugins

- Copy-as-Markdown Chrome extension Firefox add-on

More note taking and skill sharing ideas

We are keeping notes about notetaking in the Fieldnotes project's README and branches off from there. These will expand iteratively, some ideas are:

- Flash cards to learn new skills from others
- Labelling issus by topic to discover others' approaches in problem-solving
- Complex ticket follow-through after seniour help sessions

These are more fully explored within the Fieldnotes project itself. As with the Descripton templates, the spirit of this workflow is non-prescriptive: take this idea and make it your own.

SECTION: support/workflows/gitlab-com_customizations.md

Overview

Infrastructure team is responsible for driving the evolution of GitLab.com. They control and monitor our biggest GitLab instance.

GitLab.com has certain customizations specific to it. These customization are managed through the chef-repo(internal).

Limits applied to GitLab.com

These limits are subject to change without prior notice.

For Gitaly nodes, specific limits can be found under defaultattributes => omnibus-gitlab => gitlabrb => gitaly => concurrency for the different environments:

<!-- vale handbook.Repetition = NO -->
- For production production
- For canary production
- For main stage
<!-- vale handbook.Repetition = YES -->

SECTION: support/workflows/gitlab-com_overview.md

Overview

This page is meant to provide a general overview of how GitLab.com is different from self-managed instances of GitLab.

Please note that context for the following sections on this page should be covered by the various workflows that Support utilizes when working with GitLab.com along with the GitLab.com Basics training module.

GitLab.com Architecture

GitLab.com is the largest known GitLab instance. It is monitored and maintained 24/7 by our infrastructure team.

The Support team should have a general understanding of its architecture along with how to access logs (Kibana) and error reports (Sentry) to troubleshoot reported issues.

As well, Support team members should be aware that GitLab.com has certain customizations. These customization are applied through the chef-repo. Details of GitLab.com customizatons can be found in GitLab.com custom limits

Numerous Support team members also assist with incidents as CMOC.

Legal Context

When signing up, users agree to our terms, which means they are bound by them as well.

Violation of terms, including DMCA and code of conduct, are taken care of by Security Operations.

Administration

With GitLab.com, GitLab (the company) is the administrator of the instance. This has a number of consequences, outlined below.

Users Are Not Admins

Users including customers never have an admin role.

This means that none of our administrator specific documentation will apply to end-users, and instance level settings are managed by our infrastructure team.

Accounts Belong to Users

Due to the current way users register for accounts, terms apply to individual accounts and information should not be shared to others unless they are an Enterprise User as defined below.

> Note: Only share information with a user if they have access to it.

While it can sometimes make support interactions more difficult or frustrating, even something as basic as the email address on an account should not be shared if it's not public, or the user has not provided us explicit permission to share it with the other individual.

Enterprise Users

As of 2021-02-01 when our terms were last updated, we introduced the definition of an enterprise user.

Enterprise user accounts belong to the company that purchased a GitLab subscription. This means when requested by an Owner in the top-level of a paid group, information can be shared about, and actions can be made on behalf of an enterprise user.

To share private information or take any action, proof of account ownership is required as usual.

Enterprise users belong to a group based on the enterprisegroupid user attribute.
See the enterprise users documentation page for details on how this happens in GitLab.

For the purposes of support, a user may still be considered an enterprise user when both of the following conditions are met:

1. The user's primary email has a domain that is owned by the company of the paid group, this means one of the following is true:
   - The WHOIS information on the domain matches the organization name
   - The email domain matches the subscription holder in CDOT
   - The email domain matches that of an Owner in the top-level namespace
1. The user account meets one of the following conditions:
   - was created 2021-02-01 or later.
   - has a SAML or SCIM identity tied to the organization's group.
   - has a provisionedbygroupid value that is the same as the organization's group's ID.
   - is a member of the organization's group, where the subscription was purchased or renewed 2021-02-01 or later.

If the Owner is requesting access to an account which has a primary email in the company domain, but does not meet any of the second conditions, then we must treat the account as belonging to the user. In this case, the only recourse for the Owner to add the user's primary email as a CC on the ticket, then the user validates their own account.

The relevant information can be found in the Zendesk GitLab Super App: User Lookup, GitLab admin or API. Subscription information can additionally be found in CustomersDot.

{{% include "includes/support-quick-reference.md" %}}

SECTION: support/workflows/google_cloud_credit_troubleshooting.md

Overview

Use this workflow when user reports an inability to utilize their Google Cloud Credit (GCP Credit).

Workflow

1. Ask user if they've used the following URL when they applied for the credit: <https://cloud.google.com/partners/partnercredit/?pcncode=0014M00001h35gDQAQcontact-form>
    1. If no, ask them to try the link and report any issues.
    1. If yes, follow the below directions.
1. Ask user for the following information related to their GCP account
    1. Google Billing ID
    1. Complete name
    1. Email address
1. Submit this form on behalf of the user
    FIRST PAGE:
    1. Email Address: your (support engineer) email address
    1. What is your role? Partner
    SECOND PAGE:
    1. Your Name: user's name
    1. Email Address: user's email
    1. Company Name: GitLab
    1. Partner Type: Alliance / Technology / Vertical (excl. Healthcare) Partner
    1. Billing Account ID: user's Google billing ID
    1. Your Google Sales Rep's email: <manverma@google.com>
    1. Select SUBMIT
1. Attach a copy of the form to the Zendesk ticket for reference
1. Reply with General::Google Cloud Credit Troubleshooting Form Submitted macro letting them know that you have submitted the form.
1. Once confirmation is received, update user and close ticket.

Additional Reference

1. Reference tickets: ZD133164 & ZD117290
1. If no reply is received from Google and the user does not have any credit, contact product marketing in Slack for escalation assistance

---
SECTION: support/workflows/gpt_quick_start.md

What is GitLab Performance Tool (GPT)

The GitLab Performance Tool (gpt) is built and maintained by the GitLab Quality Engineering - Enablement team to provide performance testing of any GitLab instance. The tool has itself been built upon the industry-leading open-source tool k6 and provides numerous tests that are designed to effectively performance test GitLab.

GitLab recommends running GPT against your GitLab environment to get an effective performance test. We do not recommend running on a production instance. Only run on production if it's really required. If so, then run it at the quietest possible time. Depending on your system environment, the test may take up at least 4 hours.

NOTE: This quick start was written and adopted based on documentation for GPT v2 (2.10.0). Please always check the official GitLab Project documentation: GitLab Performance Tool for latest changes.

Requirements

- A separate workstation or server with Docker installed.
- Must be able to connect to the GitLab instance.

Initializing the environment

On your workstation with Docker installed, please run the following in your terminal:

bash
clone the gpt project
git clone https://gitlab.com/gitlab-org/quality/performance
cd performance
mkdir results

Preparing the Environment

This will generate the data that will be used for the test later. More details on GPT Project:

1. Create Personal Access Token with API scope from an Admin user.
   1. In the top-right corner on your GitLab UI, select your avatar.
   1. Select Edit profile.
   1. In the left sidebar, select Access Tokens.
   1. Enter a name and optional expiry date for the token.
   1. Select the API scopes.
   1. Select Create personal access token.
   1. Save the personal access token somewhere safe. After you leave the page, you no longer have access to the token.

1. Next, edit your environment file under ./k6/config/environment/. In this example, we will use the 2k users environment. Hence, it will be the 2k.json. Replace the values for "url" with the URL of your GitLab instance and "user" with the username of the Admin user created in the above step.

   bash
   vi ./k6/config/environment/2k.json

   Edit the following lines:

   yaml
   "url": "<your gitlab url>",
    "user": "<username that the access token belong to>",

   NOTE: If you have a top-level group named gpt, please replace the value for "rootgroup" with another unique top-level group name.

1. Run the following docker command to generate the data needed for the performance test:

bash
docker run -it -e ACCESSTOKEN=<TOKEN> -v $(pwd)/k6/config:/config -v $(pwd)/results:/results gitlab/gpt-data-generator --environment 2k.json

NOTE: Replace <TOKEN> with your personal access token created in Step 1.

Running the Tests

Full details and explanation are available at GPT project

Run the following Docker command:

bash
docker run -it -e ACCESSTOKEN=<TOKEN> -v $(pwd)/k6/config:/config -v $(pwd)/k6/tests:/tests -v $(pwd)/results:/results gitlab/gitlab-performance-tool --environment 2k.json --options <OPTIONS-FILE>.json

NOTE: Please replace <TOKEN> with your personal access token, then replace <OPTIONS-FILE> with 60s40rps.json to run against 2k users testing. [Optional] You may replace <OPTIONS-FILE> with the following recommended options files based on your target environment user count:

- 1k - 60s20rps.json
- 2k - 60s40rps.json
- 3k - 60s60rps.json
- 5k - 60s100rps.json
- 10k - 60s200rps.json
- 25k - 60s500rps.json
- 50k - 60s1000rps.json

Viewing Test Output and Results

After starting the tool you will see it running each test in order. Once all tests have completed you will be presented with a results summary. As an example, here is a test summary you can view.

For your reference, here are the test runs from GitLab:

1. Latest Results - Our automated CI pipelines run multiple times each week and will post their result summaries to the wiki here each time.
1. GitLab Versions - A collection of performance test results done against several select release versions of GitLab.'

There are known issues when running the GitLab Performance Tool. Some tests run against parts of the product which are known to be non-performant.

- Improve performance of users API under load.
- Check for other issues the Quality team has raised about other tests

For more details on other possible problems see Troubleshooting section

Cleaning Up

This step will delete the test data generated.

Method 1: Run the following Docker command

bash
docker run -it -e ACCESSTOKEN=<TOKEN> -v $(pwd)/k6/config:/config -v $(pwd)/results:/results gitlab/gpt-data-generator --environment 2k.json --clean-up

Method 2: Delete the top-level group gpt (or the unique name you've replaced at your environment json) from GitLab UI.

NOTE: There is no preference of one method over the other as both will delete the top-level group.

Reviewing results for customers

Customers often ask for their GPT results to be reviewed as part of building out a Reference Architecture.

- Check the GPT issues list if errors or issues .
- Ask for help from support team members with GPT experience.
- Alternatively reach out to either the GPT maintainers over on the gitlab-performance-tool channel on Slack.
- You can also reach out to the Reference Architecture group on their tracker and raise a Request for Help via the template if you suspect performance issues are related to the environmental design or makeup.

SECTION: support/workflows/handling_offer_migration_tickets.md

Overview

You may encounter a support ticket from a paid customer who is migrating between offerings (SM and SaaS), and who at the time of the ticket has a paid license to only one of them. For example, they're using a Self-managed instance and want to migrate to GitLab.com, and they only have an active Self-managed license (i.e. they have no paid SaaS subscription).

If the problem statement of the ticket involves the unpaid offering, that raises some issues:

- Do we support them, or is this out of scope?
- If we support them, then:
  - How do we configure the ticket so that it will be visible to SEs with the

right focus area?
- What level of support should they get?
- How do we configure the ticket to have the right SLA?
- How do we inform SEs that a decision has been made to support the customer?

In this type of situation, we do want to support the customer even though their contract does not require us to do so. Think of it this way: they are a paid customer and they're going to continue to be a paid customer, so let's give them a great customer experience rather than let technicalities get in the way.

How to make this work

1. Contact the customer to gather information
   1. First, explain that although they are only entitled to support on their current offering, we intend to support them anyway.
   1. Ask them to confirm what support level they intend to purchase on the new offering.
   1. Ask them the target date for completing the migration.
1. Open a Support Ops issue in this project:
   1. Specify that the request is to support a product offering migration.
   1. Specify the new support level to be applied to the organization.
   1. Specify the migration target date.
1. From here, Support Ops will:
   1. Add the support level tag to the org. There will then be two, so that the customer can get help on both their current and future offerings.
   1. Add text to the org notes explaining that the customer is migrating between offerings and that we have approved offering level support until target date.
1. Once Support Ops has completed their work, remove from the ticket the support level tag that does not belong. If you're not sure which one(s) to remove, ping in supportoperations for help.

NOTE: If you assign yourself to a new ticket and find an internal note indicating that it involves a product offering migration, please take the final step described above: remove the support level tag that doesn't align with the ticket's problem statement.

For details on the Support Ops side of the process, please see Setting Up an Organization for an Offering Migration.

SECTION: support/workflows/handling_sales_info_requests.md

Overview

Use this workflow when there's a query about new GitLab offerings from an existing customer. It can also be used to handle situations where customers express their

need via a support ticket to get in contact with their account manager.

Workflow

1. Check the organization's information in Zendesk to find the CSM for the customer.
   > Note: In Zendesk, click on the organization name under the tab at the top,
   then scroll down and look for the Account Owner or Customer Success Manager fields.
1. Respond to the ticket, CC-ing the CSM letting them know of the
customer's interest in new/more GitLab offerings.

SECTION: support/workflows/handling_upgrade_renewal_requests.md

Overview

Use this workflow when there's a query about upgrading or renewing a GitLab
license or GitLab.com subscription.

Workflow

1. Make sure that there aren't any other types of queries that would need the Support team's attention
1. Use the General::Forms::Incorrect form used macro to transfer the ticket

High Priority Cases

In some cases, a renewal may be of particularly high priority and needs urgent attention (e.g. the license
will expire within a few days). In these cases, please:

1. Determine who the account owner is in SFDC
1. Ping them on Slack with the appropriate ticket number and context

---
SECTION: support/workflows/high-priority-all-regions-tickets-workflow.md

The High-Priority All-Region (HPAR) ticket handling process has been deprecated. The actual need
for coordinated round-the-clock effort on a ticket is infrequent enough that it can be handled by
exception.

If a customer requests for round-the-clock attention on a ticket, or if you feel such attention would be beneficial to deliver results for a customer, please raise a Support Ticket Attention Request
to start a discussion with the Support Manager On-Call.

SECTION: support/workflows/how-to-create-npm-package.md

Overview

The purpose of this workflow is to make it easy(or easier) to test common customer scenarios by having access to package(s). You can follow the steps outlined below
to create a package, or use this one. Just make sure to reference to correct project ID when publishing your package (Step 3).

Creating A Scoped Package

- A scoped package is a package that's available under a scope(insert you don't say meme). The scope is used in the namespace, and usually either references
the org, or your username, depending on where the package is hosted. For example, if I create a package, and host it under my GitLab account, the scope will be @sahbabou/name-of-my-package

NOTE: We do not currently support publishing packages from subgroup/project

For the purpose of this workflow, our package will not do anything special since it will not have any functionality other than to confirm it's been installed. As a result, we will
only need a package.json file.

Step 1: Create a directory for your package

- mkdir npm-package

Step 2: Initialize the project on git

- git init
- git remote add origin <yourgiturl>

Step 3: Navigate to the npm-package directory and initialize the npm package under your org/username

This will create a package.json

- npm init --scope=<yourgroup/username>

Step 4, follow the steps in the prompt

You don't need to add a license, keyword etc. All that matters it that:

- The package name has the right scope
- It points to the correct git url
- If it asks for a command, you can pass npm start, or else it'll just show an error command in the package.json, which for now is harmless

NOTE Note: You can always change the values in the package.json

1. Create and provision the .npmrc file

The npmrc file is one of the locations where npm gets its settings. In our case, the configuration, per the official GitLab NPM Registry docs, it will hold is where to look for packages under a particular
scope, as well as the current user's authentication to push/pull packages.

Step 1: Create the ~/.npmrc file

- The ~/.npmrc generally lives in the home directory, however, sometimes, it can be created within the project's root directory, but also has no effect on packages that
are installed globally (npm install -g)

Step 2: Provision the ~/.npmrc file

- Add the registry to your scope, that way npm knows where to look for packages that start with @nameofyourscope.
  Example:
  - @sahbabou:registry=https://gitlab.ahbabou.com/api/v4/packages/npm
- Add your Oauth token(create one here if you don't have one), and add it using this format:
  - //gitlab.ahbabou.com/api/v4/packages/npm/:authToken=<Oauth Token>

NOTE Note: This is so you can pull packages from that repo when running npm install.

- Add the auth token to the URL to publish your package:

  - //gitlab.ahbabou.com/api/v4/projects/<projectid>/packages/npm/:authToken=<Oauth Token>

NOTE Note: We can't install nor publish this particular project yet, because we still have to tell our package where it can publish itself.

3. Publishing The Scoped Package

- Add this to your package.json:

  `"publishConfig": {
     "@yourscope:registry":"<yourgiturl>/api/v4/projects/<projectId>/packages/npm/"
   },`

- Run npm publish
- Check the Packages section in your package's repository to confirm that it has been published

your-gitlab-url/package-project/-/packages

4. Pulling the scoped package from the NPM Registry on GitLab

- You can run npm install from any other npm project(including this one), or clone this project, then run npm install from the project's directory, and then
confirm whether you're able to install the package.

5. Common troubleshooting tips

Note: NOTE: If on < 11.9, packages with a dot in the package name return a 403 on npm install @test.group/project is read as @test. The workaround is to either:

- Rename the groups/username so they don't include a dot in the URL(sara-ahbabou/saraahbabou)
- Re-publish packages under the new scope. Be sure to update the publichConfig value in the package.json:

```
`"publishConfig": {
    "@username-dash:registry":"http://localhost:3001/api/v4/projects/24/packages/npm/"
}`
```

- Update the .npmrc file so it reflects the new group name/username(@sara-ahbabou:registry or @saraahbabou:registry)

If you receive a 40x error, re-run the same command with a --verbose flag, and confirm whether it's hitting the correct registry

- 401: Make sure there's no trailing / in the URL in your ~/.npmrc
- 403: Make sure you're using the correct token with the appropriate permissions
- Also worth double checking whether the URL to your git repo contains http/https, especially if/when self-managed
- Make sure you're using a Premium/Ultimate license, especially if you're using an older license :)

6. Publishing a package to the NPM Registry on GitLab via GitLab CI

In order to run a job to publish your package in a job, paste the snippet below in your gitlab-ci.yml. For the job to be successful, you will need to:

1. Run it from the package's repository
1. Add your Personal Access Token and Oauth Access Token as environment variables(this scripts assumes you named them PERSONALACCESSTOKEN and OAUTHTOKEN)
1. Add the following lines in package.json in script so the commit message includes the package version and name:

```
text
"scripts": {
      "getName": "echo  $npmpackagename"
      "getVersion": "echo  $npmpackageversion",
    }
```

```
text
image: node:latest
```

This template assumes you have the following settings configured
OAUTH Access Token generated and added as an Environment Variable under Project -> Settings ->

CI/CD https://docs.gitlab.com/api/oauth2/
Personal Access Token added as an Environment Variable in order to update your package.json with the new version value
Your package.json contains the path to your private NPM Registry on GitLab. https://docs.gitlab.com/user/packages/npmregistry/uploading-packages

```
cache:
  paths:
    - nodemodules/

build:createnpmrc:
  stage: build
  script:
    - |
      if [ ! -f .npmrc ]; then echo .npmrc missing. Creating one now. Please review the following link for more information https://docs.gitlab.com/user/packages/npmregistry/authenticating-with-an-oauth-token;
          export NPMPROJECTURL=$(echo "$CIPROJECTURL" | sed "s/${CIPROJECTPATH//\//\\/}/api\/v4/g")
          export NPMREGISTRYPATHS=$(echo "$CIPROJECTURL" | sed "s/${CIPROJECTPATH//\//\\/}/api\/v4/g")
          export NPMRCURL=$(echo "$NPMREGISTRYPATHS" |  sed 's/[^:][:]//')
          export NPMRCINSTALLURL=$(echo "$NPMRCURL/packages/npm/:authToken=$OAUTHTOKEN")
          export NPMRCPUBLISHURL=$(echo "$NPMRCURL/projects/$CIPROJECTID/packages/npm/:authToken=$OAUTHTOKEN")
          export NPMSCOPE=$(echo "$CIPROJECTPATH" | sed 's/[/].$//')
          echo "@$NPMSCOPE:registry=$NPMREGISTRYPATHS/npm/" >> .npmrc
          echo "$NPMRCINSTALLURL" >> .npmrc
          echo "$NPMRCPUBLISHURL" >> .npmrc
      fi
  artifacts:
    paths:
      - .npmrc
    expirein: 1 week

buildpackage:
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client -y )'
    - eval $(ssh-agent -s)
    - ssh-add <(echo "$GITSSHPRIVKEY")
    - git config --global user.name "${GITLABUSERNAME}"
    - git config --global user.email "${GITLABUSEREMAIL}"
    - mkdir -p ~/.ssh
    - ssh-keyscan my-gitlab-machine >> gitlab-known-hosts
    - cat gitlab-known-hosts >> ~/.ssh/knownhosts
  script:
    - npm version minor --git-tag-version=false
    - npm publish
```

```
    - git status
    - git add package.json
     You can add a getName and getVersion key under scripts in your package.json to return the
name and version of your package and add it to the commit message

     "scripts": {
        "getVersion": "echo  $npmpackageversion",
        "getName": "echo  $npmpackagename"
      },
    - git commit -m "[ci skip] updated $(npm run getName -s) version to $(npm run getVersion)
    - git push -o ci.skip
http://<USERNAME>:<$PERSONALACCESSTOKEN>@gitlab.example/username/projectname.git HEAD:m
  dependencies:
    - build:createnpmrc


---
## SECTION: support/workflows/how-to-get-help.md

Getting Help on a Ticket

When working on tickets, collaboration is critical, especially when troubleshooting complex
issues, or technical areas of focus that fall outside of your experience level. Asking for help
means having a low level of shame, and also shows that you are putting the customer first
because you are working towards resolving their problem.

 Ask good questions

Asking a good question is key to getting the help you need. When in doubt, ask yourself:

> Does my question contain all the information that a reader needs to formulate a good
response?

A question should include necessary information that helps the reader get up to speed without
switching contexts. It
should also include a clear, direct ask. When asking questions, think about the following:

- Does the question have a brief, descriptive summary of the problem?
- Is there a link to the ticket, issue, docs page, or other relevant resource?
- Are there any error messages or behaviors of note that should be included?
- Is the ask clearly stated at the beginning or end of the question?

Starting with a problem summary or overview lets the reader learn about the situation without
requiring them to go
somewhere else, like a ticket. Including a link to other material helps the reader gather any
additional information
that they might need, like customer details or log files. Highlighting specific error messages
or behavior narrows the
scope. Clearly stating your ask at the beginning or end of your question makes it easy for the
reader to identify
```

what you need help with.

These guidelines are a reference, and not a requirement for asking questions. They serve as a starting point to
guide you, but don't let them block you from asking for help.

As a final opportunity for reflection, imagine that a GitLab customer is asking you this question. Is there enough
detail for you to begin formulating a response, or would you need to gather more information?

Often, writing a question out functions like rubber duck debugging.
That is, the act of thinking through the items above might highlight missing info, cause you to research something you
hadn't before, or even lead you right to the solution.

{{% alert color="info" %}}
While more information is helpful, including too much supporting detail can be confusing. Provide enough information to
satisfy the questions above in your initial post; additional details, such as logs or screenshots, can always be added
to the thread!
{{% /alert %}}

How to Get Help Workflow

If you are stuck on a ticket, the following workflow seeks to help Support Engineers realize and utilize all of the resources available to progress a ticket to resolution. This workflow lists some common resources, you can lean on to get the help you need.

If you're stuck on a ticket, first identify what's causing you to get stuck. Some examples are:

- I don't have the right knowledge to progress this ticket.
- The customer's query is out of scope, but they expect us to resolve this.
- There is a deep technical issue which needs a development expert's consult.

Then consider these options to help unblock you. And remember that escalating to unblock
is an operating principle of Results.

Bring the ticket before a group of peers

Other Support Engineers are a great resource to help out with tickets. To get help from peers, you can try one or more of the following:

1. Attend crush or help sessions such as those noted below (see the GitLab Support calendar for times):
    - AMER Senior SE Help Sessions
    - APAC/AMER or EMEA/AMER crush sessions
    - APAC or EMEA crush / collaboration sessions
    - Senior Support Office Hours (varying times)

1. Ask for help in one of the broader
   Support Slack channels.

 Expand to Support Pods and other subject matter experts

You can also do one or more of the following:

- See if there is a Support Pod that covers the area your ticket is in and ask one of the Pod members for help.
- Ask an expert within Support. You can check the Skills by Subject
  Support page to see who might have the skills to assist, or reach out
  to the Support Stable Counterpart
  for the appropriate product area. Mention those
  people in the thread and in the ticket to let them know you think they can help.
- Reach out to development teams directly in their Slack channels if you have a straightforward,
  specific question that might be quickly answerable. Don't hesitate to engage development teams
  early in the process if their expertise could help resolve an issue more efficiently. Be
  respectful of their time - the development team may direct you to open a formal request for help issue
  if your question requires more detailed information or investigation.
- Request help from the relevant GitLab Development Team.
  Gather what information you have and fill in as much detail as possible for the dev team in the issue. To get more attention, you can post in the relevant group Slack channel with a message and link to the issue.
  If you don't get a response within the SLO, contact the listed engineering manager in the project readme.
  See below for more details.
- If you have a reproducible issue, then go straight to a bug issue in the appropriate GitLab product tracker.

 Bring the ticket to managers

Especially if you feel you're stalled on a ticket and need assistance identifying next steps:

1. Always feel free to reach out to any available manager (such as your manager, or the Support Manager On-call in the supportleadership channel. They will help you to determine next steps.
   - Avoid messages with no identified DRI for responding in supportleadership as they can be missed or be a victim to the bystander effect.
1. Open a STAR in situations where getting help is urgent and important because:
   - the customer has expressed unhappiness with the service we're delivering via the ticket
   - the support engineer has noticed a correlation between several of a customers tickets that could use a more cohesive response
   - there is an urgent need for action in a different region (for example, finding a ticket owner or scheduling a call)

 How to formally Request Help from the GitLab Development Team

To enhance collaboration between the support and development teams, GitLab has implemented the

Request for Help (RFH) process. This allows support engineers to formally request assistance from the specific GitLab development groups responsible for the relevant functionality when facing technical challenges that impede ticket resolution. This section outlines the necessary steps to effectively utilize the RFH process.

It's important to note that this process is part of a broader, iterative strategy aimed at deploying this workflow across all development sections and groups at GitLab. If an RFH template for a particular development group is not yet available, please reach out to John Lyttle to initiate the creation of the required RFH template.

Are you requesting for Dev help too soon?

NO! To drive this point home, here's what one of our Developers Chad Woolley had to say about this:

> If both Support and the Customer are confused about what the next steps are, at a minimum it's an indicator that something is lacking, either in our docs or support processes, and this is an opportunity to improve those areas.

How to find the correct Development Section and Group to reach out for help

The easiest way to determine the correct place for a Support Request for Help issue is to use the docs pages. One possible workflow is as follows:

1. Locate a documentation page for the feature or topic on which you need help.
1. Scroll down to the bottom of the page and click on either the "View page source" link.
1. This will open up the .md source file of that docs page, which contains both the stage and group responsible for it noted on the top.
1. Now go to the Product Categories handbook page and search for the Development Section to which the group identified on the previous step belongs to.
1. Use the table and workflow below to create a Request for Help issue in the project identified above.

Alternatively, if you have set up the Support dotfiles, you can use the glsrequestforhelp command to quickly retrieve the "New issue" link with the correct issue template.

NOTE: A video recording of a similar workflow as the one described above can be found in the Support Training repository

The Request For Help Landing Page

The Request for Help landing page consolidates all GitLab Development Sections and their respective groups into a single repository. Unlike the previous process, all RFH templates are now centralized within this repository for easier access and management. To submit an issue, navigate to the page, locate the relevant GitLab Group, and click on its "Associated Template."

Change Process:
If a section or group has been recently modified, please create a confidential issue in the GitLab project tracker, assign to @jlyttle and notify the relevant Slack channel.
For additional details, refer to the support epic 222 for more information.

Research prior to opening an issue

Use the following repositories and resources for identifying similar issues or requests:

1. Zendesk for related tickets.
1. Previous GitLab Application bugs and feature requests.
1. Previously submitted GitLab.com issues.
1. Previously submitted Request for Help issues.
1. Discuss with knowledge experts and support stable counterparts.

Create a detailed issue

1. Open the Request for Help landing page.
1. Review the Development groups and corresponding templates section.
   - Use the provided GitLab Handbook Section Breakdown link at the bottom of the Project ReadMe file if you are unsure about which Section Sub Group and corresponding template to use.
1. Make sure to use the correct corresponding issue template when creating a new issue.
1. Complete all the fields in the issue template and attach all necessary files.
1. Ensure that an appropriate severity is set as defined by the support impact. You should set the appropriate label severity::1, severity::2, severity::3, severity::4 so that it corresponds with the priority level in Zendesk.
1. If the Zendesk ticket is escalated then add the label Support::escalated.
1. Add a 'Customer Impact' statement if necessary, advocating for the customer.
1. Ensure to follow any instructions on the template itself, such as who to assign the issue to (if not automatically assigned).
1. After creating the issue:
   - Add its link to the Zendesk ticket as an internal note and to the ticket field named GitLab Issues.
   - Use Duo Chat on the issue to identify any further details/context you should share additionally. You can use a prompt like this:
     > This issue is a request for help from the customer support team to Engineering. Identify any context that has not been shared, but that would be useful for Engineering to help provide a solution for this issue.

Tips on getting timely responses

1. Review the Opening a request for help to ensure all steps were covered.
1. Mention the engineer who is helping or assigned with every comment where you need them to review or respond.
1. If an issue is moved to another group (through a label change or moving to another project), check the corresponding template for the new group to see who to assign or mention in a comment.
1. When linking to Kibana, also upload a copy of relevant entries, a screenshot of the graph, etc. as logs rotate out after 7 days. If possible, also link to the relevant Sentry entry.
1. Many teams do not have access to customer information. So make sure if you are accessing information using elevated access (Such as GitLab.com Admin) that you provide information in the issue directly that may be required to understand the problem.

When Development Requires Additional Information

Development engineers may apply the RFH::Needs more info label to your issue if additional details are needed. This label indicates that your issue requires clarification or additional information to proceed. When you see this label:

1. Review any comments from the development team carefully.
1. Provide all requested information in a new comment.
1. Mention the relevant developer in your response.
1. Ensure you've included a detailed description of the problem, the required logs, screenshots, or the required reproduction steps.

Once you've provided the requested information, the development team will remove the RFH::Needs more info label and continue with the investigation.

Please Note: Providing thorough and prompt responses when this label is applied helps ensure your issue can be investigated and resolved efficiently.

Escalate to unblock a request

If you encounter any problems, such as obtaining a timely response from Development, then please take one or more of the following steps:

1. Check if the engineer(s) assisting with the request is on PTO through Slack or their GitLab status. If they are on PTO, mention the contacts (listed in the issue template) in the issue, or their backups if they are also on PTO, requesting an update via the issue. Backups are listed in a coverage issue, or in Slack.
1. Make the corresponding Development group Engineering Manager aware by mentioning them in the issue. You can identify the relevant Engineering Manager by checking the Development Group Handbook Page from each Projects Readme Section which provides a section named Development Groups with their corresponding templates and labels.
1. Feel empowered to ping the contacts and/or Engineering Manager in the corresponding product/development group Slack channel along with a link to the issue, requesting an update.
1. Reach out to a Support Engineering Manager for further guidance directly or in the supportleadership Slack channel.

Closing: Document and knowledge share

1. Once you receive all the necessary assistance, ensure to close the issue and add a comment explaining why it is being closed.
1. Consider whether the solution or information contained within the issue can be used to update the GitLab documentation.
   1. If you have updated the GitLab documentation, add the label documentation::created and link the merge request.
   1. Otherwise, add the label documentation::candidate, and create a GitLab issue to update the relevant documentation.

Quick Links and Resources

- Needs Collaboration view in ZenDesk.
- Create a Support pairing session issue.
- Support Workflows to follow relevant troubleshooting workflow.

- Support Documentation links for quick references to helpful GitLab documentation.
- Skills by Subject to find a Support Engineer scoped to the skill set needed for help.
- DevOps Stages to find the right development or product team to reach out to.
- Emergency runbooks with troubleshooting tips, even if not an emergency.
- See which manager is on-call if guidance is needed on something urgent.

General Troubleshooting Resources

Every problem is a little bit different. Sometimes it makes sense to try a different troubleshooting technique. These resources talk about general purpose approaches to troubleshooting:

- Julia Evans' comics, especially the ones about debugging
- The Pocket Guide to Debugging (PDF)
- General Purpose Troubleshooting Principles

Request for Help Lifecycle diagram

```
mermaid
stateDiagram-v2
    [] --> IssueOpened: Issue Created by Support
    IssueOpened --> Active: Add Issue Opened Label
    IssueOpened --> SupportTriage: Add Triage by Support label

    state SupportTriage {
        SupportAuthor --> ExpertReview: Support author responds
        ExpertReview --> SupportAuthor: Support expert responds
    }

    SupportTriage --> Active: Needs Dev Team Input

    state Active {
        SupportComment --> DevComment: Add Last comment from support team label
        DevComment --> SupportComment: Add Last comment from dev team label
        DevComment --> NeedsInfo: Add Needs more info label
        NeedsInfo --> SupportComment: Support provides info
    }

    Active --> PendingClosure: Inactive 14d

    state PendingClosure {
        [] --> Inactivity: Add Pending-Closure label
        Inactivity --> AutoClose: After 7d
    }

    PendingClosure --> Active: Remove Pending-Closure label
    PendingClosure --> Closed: Add Issue-Closed label
    Active --> Closed: Resolution found and/or Issue Closed
    SupportTriage --> Closed: Resolution found
```

```
    state Closed {
        SendReminders --> Resolved: Add Doc-Reminder label Add Resolution-Type label
    }

    Closed --> []: RFH Lifecycle complete
```

SECTION: support/workflows/how-to-respond-to-feedback.md

To understand the factors contributing to Support Satisfaction,
we review feedback received for support tickets. Issues are automatically
created in the Feedback issue tracker
(internal only) for all Support Satisfaction feedback received and assigned to the manager of the person who assigned to the ticket.

NOTE: The following categories of tickets do not receive surveys:

- Spam tickets (ones marked as spam and suspended)
- Free user tickets (ones containing the tag freecustomer)
- Embargo tickets (ones containing the tag comembargo)
- Tickets with the organization GitLab or DigitalOcean Support

What the customer receives

When a ticket is solved the customer receives an email inviting them to complete a survey by answering one simple question:

- Thinking about your recent experience of creating and working on a ticket, how easy was it to work with GitLab Customer Support?

Choice of rating range from Extremely Easy to Extremely Difficult with 5 intermediate options. The resulting feedback issue will identify these as ratings 7 down to 1

An optional second question asks:

- What would make working on a ticket with GitLab Customer Support easier?

If the customer opts out of completing this section the resulting feedback issue will contain 'User did not leave a comment'

Examples of forms are located here

Subscribing to Customer Feedback Issues

By territory

If you'd like to subscribe to CESs submitted by customers from a certain
territory, you can subscribe to the appropriate OrganizationRegion scoped
label through the Feedback project labels page.

These labels are applied based on organization information synced to Zendesk from SFDC.

Label	Description
APAC	Asia-Pacific
EMEA	Europe, Middle East and Africa
LATAM	Latin America (includes all of Central & South America)
NORAM	North America
NCSA	North, Central, South America (legacy region being phased out)
Unknown	Unknown

The single source of truth for these definitions can be found in the Go to Market Glossary handbook page.

Who is responsible for reviewing Support Satisfaction feedback

Each Support Engineering Manager is responsible for reviewing and actioning feedback for their reports. Issues will be directly assigned to managers, and should be addressed within 7 days.

This CES data will be reviewed by Senior Leaders and presented in Monthly region reviews.

CES

The manager of the person to whom a ticket is assigned is responsible for reviewing customer feedback on that ticket. Feedback issues
are assigned to the managers automatically. The manager receives email notification from GitLab and a To-Do item.

Mid-ticket feedback

Support Managers On-call are responsible for initial triage of mid-ticket feedback received during their shift. Notifications are posted in the supportticket-attention-requests channel.

Sources of feedback

Currently, the following methods create feedback issues for review:

1. Automatic email survey -- sent to customers when tickets are solved.
1. Mid-ticket feedback link -- each Public Comment from a GitLab Support Engineer or Manager has a link to a form where a customer can provide feedback or request contact from a manager while the ticket is open (introduced in issue 2913).
    1. This feedback form creates issues in the customer feedback project, with a subject format of Positive/Negative/Neutral feedback for ticket nnnnnn, and is automatically assigned to the Support Manager On-call and the manager of the Support Engineer assigned to the ticket.
    1. If the feedback is negative, there is an option to request manager contact (within 48hrs Mon-Fri). If this option is chosen, a Slack notification is sent to the supportticket-attention-requests channel. The On Call Manager should promptly follow the guidance in Handling mid ticket feedback requesting manager contact during business hours.
1. GitLab team members (such as CSMs and Sales team) can open an Indirect Feedback issue with details they received from the customer.

1. Any issue requiring contact can also be identified by applying the SSAT::Contact label. In the Description or in a Comment, specify that manager contact was requested.

What does success look like?

Within 7 days, for each of your Support Engineers who receive feedback:

1. You should have performed the triage work described in the handling "Good" and "Bad" sections
for each feedback issue assigned to you.
1. You should have initiated any customer or GitLab group contact.
1. You should have closed all feedback issues assigned to you that have no outstanding dependencies on other groups or on the customer.

How fast do I need to respond?

Currently there is no SLA for responding to Feedback Issues, but if you follow the process defined on this page, you should send an initial response to each issue within 7 days of its creation.

Workflows for reviewing feedback

Our Feedback and Complaints handbook page provides general guidance on assessing and responding to feedback.

Handling "Good" Reviews - Customer Effort Scores 5,6 or 7

For each feedback issue labeled "satisfaction::good":

1. read through the feedback and check for anything actionable - sometimes customers provide really good actionable feedback in positive reviews
1. consider sharing the feedback in the Support Week in Review document (see below)
1. if no further action is needed, /close the Feedback Issue.

Note: Support Engineers can see the Feedback comments on their tickets, and get notified by Zendesk when a Feedback comment is added. You do not need to notify them or their managers.

Note: After 7 days of inactivity, the GitLab Support Bot closes "satisfaction::good" issues.

Sharing positive feedback in Support Week in Review (SWIR)

To share positive feedback in the Support Week in Review, each week an issue will be created in the Support Week In Review Tracker and tagged with ~"SWIR::SSAT".

Anything you add to the body of this issue will be included in the SWIR digest for the week. No further action is required other than having the body of the issue updated. Please do be aware of some considerations in formatting feedback in the SWIR issue.

You only need update the SWIR issue for feedback you want to be sure makes it in. SWIR hosts will select some positive feedback each week to share with the team.

Due Date: the cut off for content for the SWIR is close of business on your Thursday. Plan to add any ticket feedback before this time. Anything you want to add after this time needs to be added to the content for the following week, to ensure it is included in the audio recording.

Guidance for SWIR Hosts

Each week, take note of any positive feedback included by managers in the ~SWIR::SSAT issue and be sure to include it.

When selecting additional feedback to share, you don't need to share every piece of positive feedback. Consider the following when choosing what to share:

1. Comments that stand out to you - the ones that you dwell on and smile as you read them
1. The customer has taken the time to name the individual(s) they appreciated
1. The customer has described why they were satisfied or how our support improved their day (we get some great stories!)
1. If there are general sentiments or themes, feel free to congratulate the whole team. For example, if we were praised for our overall approach to support, speed, clarity, etc.
1. Is the feedback definitely positive? Sometimes comments in positive feedback can be neutral or even negative. For example "I would have liked a quicker response", or "I was satisfied" are valuable to us, but they don't really encourage the team when shared in the SWIR.

Formatting feedback in SWIR issue

When adding the comment to the CES issue in the support-week-in-review tracker, feel free to use markdown formatting. If you wish to use headers () please

- use H4 () or lower
- be aware that headers will be included in the table of contents in the issue

Generally, include the ticket number with a link to the ticket, the comment from the customer, and where applicable @ mention the person (or people) who primarily worked the ticket.

Automatically collecting positive feedback

The populatessat job in the support-week-in-review tracker will automatically collect open issues labeled with ~"satisfaction::good" and append a nicely formatted version to the open CES issue.

To run this job:

1. [Run a manual pipeline] (https://docs.gitlab.com/ci/pipelines/run-a-pipeline-manually) and run the populatessat job.

You can safely re-run this task as many times as you'd like as it will append to the issue.

Handling "Bad" Reviews - Customer Effort Scores 1,2,3 or 4

For feedback issues labeled "satisfaction::bad", click through to the ticket, and review it to determine the following:

1. Why the feedback was given
1. If further action needs to be taken
1. If the customer should be contacted to discuss the feedback given
1. If the feedback should be discussed with a CSM

You should document your finding and any follow-up actions taken in the issue.
You may use the following template to add a comment to the feedback issue (NOT the ticket!):

text
 Summary of ticket/feedback:
 Action to be taken:
 Contact customer to discuss feedback? (Y/N)
 Make the CSM aware of this feedback? (Y/N)

The above text will be automatically added as a comment to "bad" reviews. You might also consider adding the above snippet to a comment template for quick use.

If no action needs to be taken, and the customer does not need to be contacted to discuss the ticket, /close the Feedback Issue.

When you see "bad" feedback on an apparently successful ticket

Sometimes, when you review the ticket, you will see that the ticket seems to have been resolved successfully. The customer may have even said something like "thank you, you can close the ticket". We strongly encourage you to contact the customer when this happens. If the ticket is still in Solved (but not Closed) status, the customer can change their rating, if they did not mean to mark it "bad".

Understanding the customer's reason

Many customers do not provide a reason for the "bad" review they submit. Even if a reason is provided, it may not be clear why the customer was dissatisfied. The reviewing manager should carefully review the ticket and seek to understand why a bad review was given. If necessary, contact the customer to learn more.

Once the reason behind the "bad" review is understood, apply the feedback scoped label that best describes the situation:

Label	Description
~feedback::customer-resolved	The customer resolved the ticket
~feedback::docs-new-issue	Encountered Documentation problem; new Issue and/or MR filed
~feedback::docs-not-helpful	Documentation unhelpful to customer and/or Support Engineer
~feedback::known-issue	Known issue with Issue already created
~feedback::new-product-issue	Encountered Product problem; new Issue and/or MR filed
~feedback::missing-info-from-customer	Customer did not supply enough information for

investigation |
~feedback::missing-info-by-engineer	Support Engineer did not provide adequate information to customer
~feedback::outside-support	Problem internal to GitLab but not directly by Support
~feedback::process-confusing-customer	Customer found support process confusing or unclear
~feedback::process-engineer-not-followed	Support Engineer did not follow documented support process
~feedback::process-does-not-exist	Problem does not have an existing Support process
~feedback::process-sfdc	Problem caused by Zendesk-Salesforce integration (includes delays due to needs-org, incorrect contact info/matching in SFDC, unable to validate subscription)
~feedback::soft-skills	Support Engineer responses unhelpful or customer expectations mishandled
~feedback::technical-skills-engineer	Support Engineer lacks skills or permissions to troubleshoot or resolve problem
~feedback::technical-skills-customer	Customer lacks skills or permissions to troubleshoot or resolve problem
~feedback::unable-to-fulfil-customer-expectations	Support unable to fulfil the request according to the customer's expectations
~ffeedback::extended-TTR-due-to-troubleshooting	The ticket time to resolve was extended due to troubleshooting requirements of the issue
~feedback::extended-TTR-caused by-technical-complexity	The ticket time to resolve was extended due to the technical complexity of the issue

Note: For the full list of feedback labels and their descriptions, visit the labels page in the support-feedback project.

This is important to help Support understand and respond to Support Satisfaction trends.

If there is action to be taken

Determine the course of action and tag appropriate people. Note that indirect feedback received from a customer/prospect will typically have the next action chosen for us.

Examples of possible actions:

- Contact the customer (see below)
- Create a new Issue to discuss the feedback in support-team-meta (cross-link the Feedback Issue as related)
- Tag the manager of the person or people who participated in the ticket to discuss in a 1:1
- Tag a product group for awareness (some negative feedback is product related)

If further discussion is warranted, leave the Feedback Issue open. Otherwise, /close it.

If the customer should be contacted

When the customer requests contact via a mid ticket feedback request:

1. The On call manager is responsible for the follow up.

If you believe the customer should be contacted following completion of a closed ticket survey:

1. Determine the best way to reach out. Some options are:
   1. Sending the customer an email from your GitLab email address. This should be the appropriate option in the majority of cases.
   1. Responding directly on the Zendesk ticket. This is appropriate if you determine that the ticket was not adequately resolved and work should be continued.
      - If reopening a ticket, make sure that you assign and brief a support engineer (typically the existing assignee) on the next actions to be taken.
      - Note that reopening a closed or solved ticket affects measurements of reopen rate and time to resolution.
1. When reaching out to the customer, make sure you do the following:
   1. introduce yourself, describing who you are and your role at GitLab
   1. note the specifics of the situation, including ticket ID - you can include a link using the format https://support.gitlab.com/hc/requests/<ticket number>
   1. restate and validate the customer's comments
   1. offer any apologies or clarifications required, including links to relevant documentation
   1. offer your Calendly link to schedule a video call
1. Update the Feedback Issue as follows:
   1. Add the text of your email as a comment in the Feedback Issue.
   1. Apply the label ~ssat-manager-contacted-customer.
   1. /close the Feedback Issue; followup continues via email.
   1. After closing the Issue, if there are any additional actions that arise from your interaction with the customer, go back and note them in the Feedback Issue.

SECTION: support/workflows/how-to-respond-to-tickets.md

How to respond to a ticket

Smart Humans Provide Smart Support

We aim to hire smart people, and let them be smart. This means we try to offer sensible guidelines that will help, but avoid "scripts" or rigidity. Speak in your natural voice as you would to a peer at a conference. Obviously avoid unprofessional language, but you'll want to match the customer's tone. There is often a desire to "unify" by everyone speaking in a robotic tone:

> "Thank you for contacting support. We can help you with this. It looks like you are asking for help with resetting your password·"

This dehumanizes us and we lose our best asset: Support from a human. When you speak more naturally it anchors that we are also real people and not "support minds as a service:"

> "Ah, sorry to hear that you lost your password. I've issued a password reset and you are good to go. In the future, you can use this link:
>

> <link>
>
> Let us know if there is anything else we can help with. "

We Aren't a Cannery (but we sometimes use canned goods)

At GitLab, we consider the elements of our responses carefully. If you find yourself wanting to use canned responses, or are saying the same things over and over, it's probably an opportunity to improve our process. That is, instead of creating a text expander asking for logs, take a step back. Is there something earlier in the experience of opening a support ticket that we could do to reduce the need for repetitive text? There are times when it's appropriate to use formal language and canned replies, but those will be rare. Whenever we can we push towards empathy and humanity and automate and preload the process.

Consider this spectrum:

mermaid
graph LR
    A[Pure Process] ---|Request Requires| B[Higher Level Thinking]
    C[Robotic Support] ---|Tone Is| D[Voice-Empathy and Humanity]

Be empowered: at GitLab Support we want humans with agency, not agents. If something feels broken, ask.
If something feels inefficient, fix it. Everyone can and should contribute.

The Sandwich Method

When it comes to actually answering tickets, the sandwich method is a great 3 point guideline that will help you elevate your responses. A great customer reply will contain the following 3 things:

- What you need from them.
- Present a premise or hypothesis which explains your thoughts on why the requested items will help.
- An offer to continue helping.

For example, a customer might ask:

> " My GitLab server appears to be slowing down. Can you help me?"

An okay response is:

> " Please send us over your production logs and we can use that to troubleshoot some more. "

Notice we asked for what we needed, and we'll be able to help. Let's make it great using the sandwich method:

> "It will be helpful to get as many logs as possible during the slowness to help us isolate

the problem. You can find them in /var/log/gitlab (This is our ask)
>
> Usually when we see slowness, it's isolated to a specific part of the application. Can you help us narrow down the issue by outlining when you see things slow down? (This is our premise for them to reinforce our expertise.)
>
> Once you send these over and help us understand how you are getting to the slow state we'll be happy to help you dive in some more." (This is us reassuring them we'll help.)

We've asked for what we need early. Emphasize the ask early, so they can start thinking on it and if they stop reading right there, they didn't miss the ask. We've given a hypothesis which is something for them to chew on and understand our vantage. We don't want to serve our customers, we want to partner with them. This is one way for them to see us as a peer vs. "support minds as a service."

We then make sure to let them know we are still here and will be when they come back.

There will be times where you might need to add more or even apologize for something, but this method should be applicable to the majority of tickets and help us deliver excellence.

Two modes of operation: the Characterization Mode and the Hypothesis testing mode

We can think of working through tickets as alternating between two modes of operation: The Characterization Mode (CM) and the Hypothesis Testing mode (HT).

In the characterization mode, we're working to establish basic facts about what the user is trying to do, what is effectively happening, and information about context and state that might be relevant. We can construe a ticket as a puzzle as a useful metaphor, and look to elaborate its contradictions. This also might serve as a baseline for reproducing steps and for a potential bug report.

We can be transparent that we're working to characterize the user's issue. When in this operation mode, we can ask:

- What the user is trying to do
- Why the user is trying to do it
- How the system is effectively behaving
- How the user thinks the system should behave
- State and context information that might be influencing the behavior we are seeing

The second mode of operation is the Hypothesis Testing mode. This is the creative step where we can behave like scientists and theorize about what might be going on the user's side.

We also can be transparent that we're in the hypothesis testing mode. When doing so, we can clarify:

- What the hypothesis is
- How it accounts for the behavior
- How it accounts for other facts that were already established
- How it doesn't account for some facts

- How can we test it
- Whether the test involves any risk

It is interesting to note that the hypothesis test feeds back into the characterization mode, as we're establishing new facts about the user's scenario with the test.

You can come up with multiple theories and corresponding tests in a single response. In fact, doing so might help make explicit the structure of the operation modes described above. The facts established in the characterization step would be common to all theories. The possibilities described in a single theory, however, might not apply to others, so it is important to keep them separate.

The idea for this was taken fr